

Notes on Ch 27: A Field Guide to Base R

The Caveman Coder

2025-08-18

Topics covered:

- Subsetting with `[]`
- Subsetting with `[][]`
- Subsetting with `$`
- For loops
- Essential plotting functions

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Selecting multiple elements with `[]`

Subsetting vectors

```
# Accessing elements by position
x <- c("one", "two", "three", "four", "five")

x[c(3, 2, 5)]
```

```
## [1] "three" "two"   "five"
```

Since you can repeatedly access a position, the term “subsetting” is actually a misnomer:

```
x[c(1, 1, 5, 5, 5, 2)]
```

```
## [1] "one"  "one"  "five" "five" "five" "two"
```

Dropping the elements at specified positions using negative index positions:

```
x[c(-1, -3, -5)]
```

```
## [1] "two"  "four"
```

Subsetting with a logical vector:

```
x <- c(10, 3, NA, 5, 8, 1, NA)
```

```
# All non-missing values of x  
x[!is.na(x)]
```

```
## [1] 10 3 5 8 1
```

```
# All even (or missing!) values of x  
x[x %% 2 == 0]
```

```
## [1] 10 NA 8 NA
```

Unlike `filter()`, NA indices will be included in the output as NAs.

Subsetting with a character vector:

```
x <- c(abc = 1, def = 2, xyz = 5)  
x[c("xyz", "def")]
```

```
## xyz def  
## 5 2
```

Subsetting with “nothing” returns the entire vector:

```
x[]
```

```
## abc def xyz  
## 1 2 5
```

This is a feature, not a bug. Useful when subsetting 2d structures like tibbles.

Subsetting data frames

Format is `df[rows, columns]`.

Examples:

```
df <- tibble(  
  x = 1:3,  
  y = c("a", "e", "f"),  
  z = runif(3)  
)
```

```
# Selecting the first row and second column  
df[1, 2]
```

```
## # A tibble: 1 x 1  
##   y  
##   <chr>  
## 1 a
```

```
# Selecting all rows and columns x and y  
df[, c("x", "y")]
```

```
## # A tibble: 3 x 2  
##       x y  
##   <int> <chr>  
## 1     1 a  
## 2     2 e
```

```
## 3      3 f
# Selecting rows where x is greater than 1 and all columns
df[df$x > 1, ]
```

```
## # A tibble: 2 x 3
##       x y       z
##   <int> <chr> <dbl>
## 1     2 e     0.405
## 2     3 f     0.806
```

The dollar sign \$ is used to access the column name.

In many applications, a tibble and data frame can be used interchangeably, but in base R, it is important to make a distinction between them: If the target object is a data frame, subsetting a single column with [] returns a vector, and subsetting multiple columns with [] returns a data frame. If the target object is a tibble, subsetting with [] will always return a tibble.

```
# Dataframe
df1 <- data.frame(x = 1:3)
df1[, "x"]
```

```
## [1] 1 2 3
```

```
# Tibble
df2 <- tibble(x = 1:3)
df2[, "x"]
```

```
## # A tibble: 3 x 1
##       x
##   <int>
## 1     1
## 2     2
## 3     3
```

To avoid this ambiguity, we can use the argument `drop = FALSE` with dataframes:

```
df1[, "x", drop = FALSE]
```

```
##      x
## 1 1
## 2 2
## 3 3
```

dplyr equivalents

The dplyr function `filter()` is equivalent to subsetting rows with a logical vector, with the missing values excluded:

```
df <- tibble(
  x = c(2, 3, 1, 1, NA),
  y = letters[1:5],
  z = runif(5)
)
df |> filter(x > 1)
```

```
## # A tibble: 2 x 3
##       x y       z
##   <dbl> <chr> <dbl>
```

```
## 1      2 a      0.374
## 2      3 b      0.530
```

```
# same as
df[!is.na(df$x) & df$x > 1, ]
```

```
## # A tibble: 2 x 3
##       x y      z
##   <dbl> <chr> <dbl>
## 1      2 a      0.374
## 2      3 b      0.530
```

```
# same as
df[which(df$x > 1), ]
```

```
## # A tibble: 2 x 3
##       x y      z
##   <dbl> <chr> <dbl>
## 1      2 a      0.374
## 2      3 b      0.530
```

The dplyr function `arrange()` is equivalent to subsetting the rows with an integer vector created with `order()`:

```
df |> arrange(x, y)
```

```
## # A tibble: 5 x 3
##       x y      z
##   <dbl> <chr> <dbl>
## 1      1 c      0.527
## 2      1 d      0.455
## 3      2 a      0.374
## 4      3 b      0.530
## 5     NA e      0.625
```

```
# same as
df[order(df$x, df$y), ]
```

```
## # A tibble: 5 x 3
##       x y      z
##   <dbl> <chr> <dbl>
## 1      1 c      0.527
## 2      1 d      0.455
## 3      2 a      0.374
## 4      3 b      0.530
## 5     NA e      0.625
```

Both dplyr functions `select()` and `relocate()` are similar to subsetting the columns with a character vector:

```
df |> select(x, z)
```

```
## # A tibble: 5 x 2
##       x      z
##   <dbl> <dbl>
## 1      2 0.374
## 2      3 0.530
## 3      1 0.527
## 4      1 0.455
## 5     NA 0.625
```

```
# same as
df[, c("x", "z")]
```

```
## # A tibble: 5 x 2
##       x     z
##   <dbl> <dbl>
## 1     2 0.374
## 2     3 0.530
## 3     1 0.527
## 4     1 0.455
## 5    NA 0.625
```

The base R function `subset()` combines the features of the dplyr functions `filter()` and `select()`:

```
df |>
  filter(x > 1) |>
  select(y, z)
```

```
## # A tibble: 2 x 2
##     y     z
##   <chr> <dbl>
## 1 a     0.374
## 2 b     0.530
```

```
# same as
df |> subset(x > 1, c(y, z))
```

```
## # A tibble: 2 x 2
##     y     z
##   <chr> <dbl>
## 1 a     0.374
## 2 b     0.530
```

Exercises

1. Create functions that take a vector as input and return:
 - a. The elements at even-numbered positions.
 - b. Every element except the last value.
 - c. Only even values (and no missing value).
2. Why is `x[-which(x > 0)]` not the same as `x[x <= 0]`? Read the documentation for `which()` and do some experiments to figure it out.

Selecting a single element with `$` and `[[ ]]`

These two are used to extract a single element.

Data frames

`$` is specialized to access by column name, while `[[ ]]` is used to access by position or by name:

```
tb <- tibble(
  x = 1:4,
  y = c(10, 4, 1, 21)
)
```

```
# by position
tb[[1]]
```

```
## [1] 1 2 3 4
```

```
# by name
tb[["x"]]
```

```
## [1] 1 2 3 4
```

```
tb$x
```

```
## [1] 1 2 3 4
```

Both can also be used to create new columns – the base R equivalent to `mutate()`:

```
tb$z <- tb$x + tb$y
tb
```

```
## # A tibble: 4 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1    10    11
## 2     2     4     6
## 3     3     1     4
## 4     4    21    25
```

The `$` accessor is very convenient to use when performing quick summaries:

```
max(diamonds$carat)
```

```
## [1] 5.01
```

```
levels(diamonds$cut)
```

```
## [1] "Fair"      "Good"      "Very Good" "Premium"   "Ideal"
```

In `dplyr`, `pull()` takes a variable name or variable position and returns just that column, and can be equivalent to the use of accessors in the code above:

```
diamonds |> pull(carat) |> max()
```

```
## [1] 5.01
```

```
diamonds |> pull(cut) |> levels()
```

```
## [1] "Fair"      "Good"      "Very Good" "Premium"   "Ideal"
```

Tibbles

The dollar `$` accessor performs partial matching when used on dataframes:

```
df <- data.frame(x1 = 1)
df$x
```

```
## [1] 1
```

```
df$z
```

```
## NULL
```

When used in tibbles, the `$` accessor is more strict:

```
tb <- tibble(x1 = 1)
```

```
tb$x
```

```
## Warning: Unknown or uninitialised column: `x`.
```

```
## NULL
```

```
tb$z
```

```
## Warning: Unknown or uninitialised column: `z`.
```

```
## NULL
```

Here is a “joke” to remember this distinction between tibbles and dataframes:

Tibbles are lazy and surly: they do less and complain more.

Lists

The \$ and [[]] “accessors extract a single component from a list:

```
l <- list(  
  a = 1:3,  
  b = "a string",  
  c = pi,  
  d = list(-1, -5)  
)
```

```
str(l[1:2])
```

```
## List of 2
```

```
## $ a: int [1:3] 1 2 3
```

```
## $ b: chr "a string"
```

```
str(l[[1]])
```

```
## int [1:3] 1 2 3
```

```
str(l[1])
```

```
## List of 1
```

```
## $ a: int [1:3] 1 2 3
```

```
str(l[4])
```

```
## List of 1
```

```
## $ d:List of 2
```

```
## ..$ : num -1
```

```
## ..$ : num -5
```

```
str(l[[4]])
```

```
## List of 2
```

```
## $ : num -1
```

```
## $ : num -5
```

```
str(l$a)
```

```
## int [1:3] 1 2 3
```

Consider this analogy: A pepper shaker that contains sachets of ground pepper.

- `pepper[1]` refers to a pepper shaker containing a single sachet.
- `pepper[2]` refers to the same pepper shaker containing the second sachet.
- `pepper[1:2]` refers to the same pepper shaker containing the first and second sachet.
- `pepper[[1]]` extracts the first pepper sachet itself.

Exercises

1. What happens when you use `[[ ]]` with a positive integer that's bigger than the length of the vector? What happens when you subset with a name that doesn't exist?
2. What would `pepper[[1]][1]` be? What about `pepper[[1]][[1]]`?

Apply family

- The most important function in this family is the `lapply()` function, which can be used to replace every `map()` function, if desired.
- There is no base R equivalent to `across()`, but using `[]` with `lapply()` can achieve something close to it.

Example:

```
df <- tibble(
  a = 1,
  b = 2,
  c = "a",
  d = "b",
  e = 4
)

# First find numeric columns
num_cols <- sapply(df, is.numeric)
num_cols

##      a      b      c      d      e
## TRUE TRUE FALSE FALSE TRUE

# Then transform each column with lapply() and replace the
# original values.
df[, num_cols]

## # A tibble: 1 x 3
##       a      b      e
##   <dbl> <dbl> <dbl>
## 1     1     2     4

df

## # A tibble: 1 x 5
##       a      b c      d      e
##   <dbl> <dbl> <chr> <chr> <dbl>
## 1     1     2 a      b      4
```

`sapply()` is similar to `lapply()`, but the former always tries to simplify the result, hence the `s` in the name.

- The authors don't recommend using this for programming, because the simplification can fail and can give unexpected results.

- Base R provides a stricter version of `sapply()`, called `vapply()` – short for vector apply.

Example using `vapply()` instead of `sapply()`:

```
vapply(df, is.numeric, logical(1))
```

```
##      a      b      c      d      e
## TRUE TRUE FALSE FALSE TRUE
```

Example using `tapply()`, which computes a single grouped summary:

```
diamonds |>
  group_by(cut) |>
  summarize(price = mean(price))
```

```
## # A tibble: 5 x 2
##   cut      price
##   <ord>    <dbl>
## 1 Fair     4359.
## 2 Good     3929.
## 3 Very Good 3982.
## 4 Premium  4584.
## 5 Ideal    3458.
```

```
tapply(diamonds$price, diamonds$cut, mean)
```

```
##      Fair      Good Very Good   Premium     Ideal
## 4358.758 3928.864 3981.760 4584.258 3457.542
```

for loops

Basic structure:

```
for (element in vector) {
  # do something with element
}
```

The most straightforward use of `for` loops is to achieve the same effect as `walk()` – call some function with a side-effect on each element of a list.

Example:

```
# version using walk
paths |> walk(append_file)

# version using for-loop
for (paths in paths) {
  append_file(paths)
}
```

To iterate over the list of file paths, it is better to iterate over their indices, rather than over the elements. To effectively do this, the function `seq_along()` can be used. The final function becomes:

```
for (i in seq_along(paths)) {
  files[[i]] <- readxl::read_excel(paths[[i]])
}
```

Finally, to combine the list of tibbles into a single tibble, the combination of `do.call()` and `rbind()` can be used:

```
do.call() + rbind()
```

To save the results, rather than make a list and save the results as we go, it is better to build the dataframe piece by piece:

```
out <- NULL
for (path in paths) {
  out <- rbind(out, readxl::read_excel(path))
}
```

Plots

There are two main types of base plot (for data science?) are scatterplots and histograms. In base R, we can make these plots using `plot()` and `hist()`.

Example:

```
hist(diamonds$carat)
```



```
plot(diamonds$carat, diamonds$price)
```

